Adaptive Provenance Weighting for Contaminated Pretraining Corpora: A Diagnostic Framework and Practical Guidelines

Anonymous

Abstract

Large-scale web corpora increasingly contain machine-generated text of unknown provenance and proportion, degrading language model pretraining. Detect-and-reweight approaches assign instance weights $w_i = (1 - q_i)^b$ based on detector scores q_i, but the exponent b must be tuned per model via grid search over a validation set. We introduce **adaptive** b^*, which derives b online from per-batch contamination statistics $\hat{\alpha}_{\text{eff}}(b)$ subject to an effective sample size (ESS) constraint, requiring no validation set and no known contamination ratio. Our experiments on FineWeb subsets (200M tokens, $\rho \in \{0.4, 0.6\}$) with Qwen3-0.6B and Llama-3.2-3B reveal three findings. **(1)** A lightweight TF-IDF + logistic regression detector (AUC = 0.939) achieves within 0.003 PPL of a 150M-parameter ModernBERT detector (AUC = 0.986), establishing that expensive detectors are unnecessary for provenance weighting. **(2)** The $\hat{\alpha}_{\text{eff}}(b)$ framework, while producing b^* values that systematically overshoot the grid-optimal b by approximately $4\times$, serves as a diagnostic tool whose b^* trajectory exposes batch-level contamination structure. **(3)** Adaptive b^* provides a zero-cost reasonable default (recovering 70% of grid-best improvement) in standard settings, though it degrades under cross-generator contamination and domain-skewed injection. We additionally provide the first systematic comparison of loss reweighting versus sampling importance resampling (SIR) under identical weighting formulas.

1 Introduction

The proliferation of machine-generated text on the web poses a growing challenge for language model pretraining. Recent estimates suggest 20–60% of web-crawled corpora may contain synthetic content from diverse generators, with the exact proportion and provenance unknown at training time. Theoretical analyses demonstrate that even small amounts of synthetic contamination can degrade scaling laws and induce model collapse [Shumailov et al., 2024].

Drayson et al. propose a detect-and-reweight framework: train a detector to produce per-instance contamination scores $q_i \in [0, 1]$, then weight each sample by $w_i = (1 - q_i)^b$ during training via Sampling Importance Resampling (SIR). While effective, this approach requires tuning b per model through validation-based grid search—different model scales need different b values (e.g., $b = 10$ for GPT-2/SmolLM $\leq$360M versus $b = 1$ for SmolLM-1.7B), and the procedure assumes access to a clean validation set.

We propose **adaptive** b^*: an online method that derives the weighting exponent from per-batch statistics of the detector score distribution, subject to an effective sample size (ESS) constraint. Our method requires neither a validation set nor knowledge of the contamination proportion, making it applicable to web-scale pretraining where these are unavailable. We implement provenance weighting via direct loss reweighting rather than SIR, providing continuous gradient-level control.

Our investigation yields three contributions, ordered by strength of evidence:

1. **Lightweight detector sufficiency.** A TF-IDF + logistic regression detector (50K parameters, AUC = 0.939, <1 min CPU training) produces training outcomes within 0.003 PPL of a ModernBERT 150M detector (AUC = 0.986, 1.5h GPU training). The marginal return of detector quality on downstream PPL is negligible, suggesting that practitioners can skip expensive neural detectors entirely.

2. $\hat{\alpha}_{\text{eff}}(b)$ **as a diagnostic framework.** We formalize the connection between batch-level contamination statistics and the weighting exponent through $\hat{\alpha}_{\text{eff}}(b)$. While the derived b^* systematically overshoots the grid-optimal value (median $b^* \approx 0.87$ versus grid-best $b = 0.2$), the b^* trajectory across batches and contamination levels provides interpretable diagnostics about contamination structure that the original framework lacks.

3. **Loss reweighting vs. SIR.** We provide a systematic comparison of continuous loss reweighting and discrete SIR under identical weighting formulas, characterizing their gradient variance and convergence behavior differences.

As an additional practical finding, adaptive b^* recovers approximately 70% of grid-best improvement at zero tuning cost, serving as a reasonable default in standard settings—though it degrades or becomes harmful under cross-generator contamination (Section 5.6) and domain-skewed injection (Section 5.7).

2 Background and Related Work

Model Collapse and Synthetic Contamination. Training on model-generated data can cause progressive degradation of output quality, termed *model collapse* [Shumailov et al., 2024]. Even low contamination ratios degrade scaling behavior, motivating methods that mitigate synthetic content during pretraining.

Detect-and-Reweight. Drayson et al. train a detector to produce scores $q_i \in [0, 1]$ indicating the probability that sample i is synthetic. Each sample receives weight $w_i = (1 - q_i)^b$, where $b > 0$ controls downweighting aggressiveness. Their implementation uses SIR: samples are stochastically resampled proportional to their weights, effectively a discrete selection mechanism. The exponent b is selected via grid search over a validation set, yielding model-specific values ($b = 10$ for $\leq$360M models, $b = 1$ for 1.7B).

Golden Ratio Weighting. Concurrent work derives a theoretically optimal weight $w^* = 1/\varphi$ but assumes the real-to-synthetic ratio k is known, which is unrealistic for web corpora.

Domain-Level Reweighting. DoReMi and DSIR optimize domain-level mixture weights, operating at a coarser granularity than our instance-level approach. SoftDedup applies soft reweighting based on duplication, which is orthogonal to provenance-based weighting. Dynamic loss-based reweighting adjusts weights based on training loss rather than detector scores, providing a complementary signal.

Convergence Theory. The bound $d(\hat{\mathbb{P}}_t, \mathbb{P}_0) \lesssim t^{-\min\{p,\alpha\}}$ from recent work motivates the definition of α_{eff} and the derivation of b^*, though we do not claim this theoretical bridge as a contribution.

Algorithm 1 Adaptive b^* with ESS Constraint

Require: Batch $\mathcal{B} = \{(x_i, q_i)\}_{i=1}^n$, ESS lower bound $n_{\min}$, search range $[0, b_{\max}]$

1: Compute $\mathrm{ESS}(b)$ for $b \in \{0, 0.1, 0.2, \ldots, b_{\max}\}$ via Eq. (2)

2: $b^* \leftarrow \max\{b : \mathrm{ESS}(b) \geq n_{\min}\}$

3: $w_i \leftarrow (1 - q_i)^{b^*}$ for each $i \in \mathcal{B}$

4: $\bar{w} \leftarrow \frac{1}{n}\sum_i w_i; \quad \tilde{w}_i \leftarrow w_i / \bar{w}$ (normalize to mean 1)

5: $\mathcal{L}_{\text{weighted}} = \frac{1}{n}\sum_i \tilde{w}_i \cdot \ell(x_i; \theta)$

6: **return** $\mathcal{L}_{\text{weighted}}$, b^*

3 Method

3.1 Problem Setting

We consider pretraining on a corpus $\mathcal{D} = \{x_1, \ldots, x_N\}$ where an unknown fraction ρ of samples are machine-generated. A pretrained detector provides scores $q_i \in [0, 1]$ estimating $P(\text{synthetic} \mid x_i)$. Our goal: assign weights $w_i = (1 - q_i)^b$ that mitigate contamination effects, with b determined automatically.

3.2 Effective Contamination Fraction $\hat{\alpha}_{\text{eff}}(b)$

For a batch $\mathcal{B}$ of size n, we define the effective contamination fraction under weighting exponent b:

$$\hat{\alpha}_{\text{eff}}(b) = 1 - \frac{\sum_{i \in \mathcal{B}} (1 - q_i)^b}{\sum_{i \in \mathcal{B}} 1} = 1 - \frac{1}{n}\sum_{i \in \mathcal{B}} (1 - q_i)^b. \tag{1}$$

This quantity measures how much effective mass is removed by the weighting scheme: when $b = 0$, $\hat{\alpha}_{\text{eff}} = 0$ (no downweighting); as $b \to \infty$, $\hat{\alpha}_{\text{eff}} \to 1 - (1 - \rho_{\mathcal{B}})$ where $\rho_{\mathcal{B}}$ is the batch contamination fraction for perfectly calibrated q_i.

3.3 ESS-Constrained b^* Derivation

We derive b^* by maximizing the contamination removal subject to maintaining sufficient effective sample size. The ESS for weights $\{w_i\}$ in batch $\mathcal{B}$ is:

$$\mathrm{ESS}(\mathcal{B}, b) = \frac{\left(\sum_{i \in \mathcal{B}} w_i\right)^2}{\sum_{i \in \mathcal{B}} w_i^2} = \frac{\left(\sum_i (1 - q_i)^b\right)^2}{\sum_i (1 - q_i)^{2b}}. \tag{2}$$

We impose $\mathrm{ESS}(\mathcal{B}, b) \geq n_{\min}$ and solve for the largest b satisfying this constraint:

$$b^* = \max\{b \geq 0 : \mathrm{ESS}(\mathcal{B}, b) \geq n_{\min}\}. \tag{3}$$

The hyperparameter $n_{\min}$ controls the aggressiveness of downweighting. Unlike the continuous exponent b which spans $[0, \infty)$ and requires multi-point grid search, $n_{\min}$ takes small integer values ($\{1, 2, 3, \ldots, n\}$) with $n_{\min} = 2$ performing well across our experiments (Section 5.5).

3.4 Loss Reweighting vs. SIR

Drayson et al. implement provenance weighting via SIR: samples are resampled proportional to w_i before each epoch, creating a modified dataset. We instead apply weights directly to the loss function (Algorithm 1, line 5). The key differences are:

- **Granularity**: Loss reweighting applies continuous weights per gradient step; SIR makes discrete include/exclude decisions per epoch.
- **Gradient variance**: Loss reweighting introduces weight-dependent variance in gradient estimates; SIR introduces resampling variance.
- **Adaptivity**: Per-batch b^* naturally integrates with loss reweighting (weights change every batch); SIR would require re-resampling per batch, negating its data-loading efficiency.

4 Experimental Setup

Data. We use FineWeb subsets totaling 200M tokens. Synthetic data is generated by Qwen3-0.6B (self-generation setting) or Llama-3.2-1B (cross-generator setting) via continuation of real text prefixes. Contamination ratios: $\rho \in \{0.4, 0.6\}$.

Detectors. (1) **TF-IDF + LR**: TF-IDF features with logistic regression, ∼50K parameters, <1 min CPU training, AUC = 0.939. (2) **ModernBERT 150M**: Fine-tuned ModernBERT, 150M parameters, 1.5h GPU training, AUC = 0.986.

Models. Qwen3-0.6B (primary) and Llama-3.2-3B (scale validation), trained for 6103 steps on 200M tokens with batch size 4 and gradient checkpointing.

Evaluation. Perplexity (PPL) on a held-out FineWeb subset (∼4M tokens). Downstream: MMLU (5-shot) and HellaSwag (10-shot) for the 3B model. All experiments use 3 random seeds (42, 123, 456) where budget permits.

Baselines. *Uniform*: standard training without reweighting. *Grid-b*: fixed $b \in \{0.1, 0.2, 0.3, 0.5, 1.0, 1.5, 2.0, 3.0, 5.$ *Detect-and-remove*: discard samples with $q > 0.5$. *Random reweighting*: weights drawn uniformly from $[0, 2]$ (mean 1).

5 Results

5.1 Full Grid Curve Reveals Low-b Optimum

Table 1 presents the complete PPL landscape across 9 fixed-b values, adaptive b^*, and the uniform baseline ($\rho = 0.4$, Qwen3-0.6B, 200M tokens).

The grid-best exponent is $b = 0.2$ across all three seeds, with PPL improvements of 0.053, 0.065, and 0.064 over uniform (seed 42, 123, 456 respectively). The PPL curve is monotonically increasing for $b > 0.2$, confirming that aggressive downweighting hurts: at $b = 0.2$, contaminated samples retain 63% weight ($w = (1-0.9)^{0.2} = 0.63$), preserving useful signal while mildly correcting distributional shift.

Adaptive b^* (median ≈ 0.87) overshoots the grid optimum by approximately 4×. This systematic overshoot reduces the aggregate recovery ratio from the previously reported 88.6% (when $b = 0.5$ was believed to be grid-best) to 69.8% (Table 2).

Table 1: Eval PPL across weighting exponents ($\rho = 0.4$, Qwen3-0.6B, 200M tokens). Lower is better. Bold: best per seed.

b	seed 42	seed 123	seed 456
0.1	17.645	17.631	17.642
0.2	**17.637**	**17.625**	**17.636**
0.3	17.641	17.629	17.640
0.5	17.651	17.638	17.650
1.0	17.692	17.674	17.680
1.5	17.718	17.701	17.710
2.0	17.741	17.725	∼17.73
3.0	17.774	17.760	∼17.77
5.0	17.818	17.804	∼17.82
Uniform	17.690	17.690	17.700
Adaptive b^*	17.650	17.643	17.660

Table 2: Adaptive b^* recovery ratio relative to grid-best ($b = 0.2$). The gap between adaptive and grid-best (0.013–0.024) exceeds cross-seed variance (∼0.007).

Seed	Uniform	Grid-best (b=0.2)	Adaptive	Ratio
42	17.690	17.637	17.650	75.5%
123	17.690	17.625	17.643	72.3%
456	17.700	17.636	17.660	62.5%
Aggregate	—	*0.182*	*0.127*	*69.8%*

5.2 Lightweight Detector Sufficiency

Table 3 compares the TF-IDF + LR and ModernBERT detectors. Despite a large AUC gap (0.939 vs. 0.986), the PPL difference is negligible: 0.003 absolute, <0.02% relative.

The monotonicity of $w = (1 - q)^b$ guarantees that ranking correctness matters more than calibration accuracy. TF-IDF + LR, despite higher calibration error (ECE 0.048 vs. 0.022) and a more diffuse q-score distribution (mid-range ratio 0.463 vs. 0.28), preserves sufficient ranking quality for effective reweighting. The practical implication: a detector trainable in under one minute on a CPU suffices for provenance weighting.

5.3 Higher Contamination ($\rho = 0.6$)

Table 4 compares reweighting effectiveness at 40% and 60% contamination. At $\rho = 0.6$, uniform PPL rises from 17.69 to 17.85 (+0.9% relative), confirming that higher contamination degrades training quality. However, adaptive improvement does *not* scale with contamination: Δ_{adaptive} decreases from 0.04 to 0.03.

The adaptive mechanism responds directionally: b^* median increases from 0.87 to 1.08 as contamination rises, reflecting higher batch-level $\hat{\alpha}_{\text{eff}}$. However, this response exacerbates overshoot—the true optimal b likely remains near 0.2, so the increased b^* at $\rho = 0.6$ moves further from the optimum. At $\rho = 0.6$, grid $b = 0.5$ achieves PPL = 17.80, yielding $\Delta = 0.05$ versus adaptive's 0.03, for a ratio of only 60%.

This result has diagnostic value: the b^* shift from 0.87 to 1.08 correctly signals increased contamination, even though the derived value is not numerically optimal. The $\hat{\alpha}_{\text{eff}}$ framework thus

Table 3: Detector quality vs. training PPL ($\rho = 0.4$, Qwen3-0.6B, adaptive b^*, seed 42). AUC improves by 5% relative, but PPL gain is negligible.

Detector	Params	AUC	ECE (cal.)	Adaptive PPL	Cost
TF-IDF + LR	∼50K	0.939	0.048	17.650	<1 min CPU
ModernBERT 150M	150M	0.986	0.022	17.647	1.5h GPU
Δ (AUC $\uparrow$ 0.047)				−0.003	

Table 4: Contamination level vs. adaptive effectiveness (Qwen3-0.6B, seed 42). Higher contamination does not amplify the benefit of adaptive reweighting.

ρ	Uniform	Adaptive	Δ_{abs}	Δ_{rel} (%)	b^* median
0.4	17.690	17.650	0.040	0.23	0.87
0.6	17.850	17.820	0.030	0.17	1.08

serves as a contamination *thermometer*, not an optimizer.

5.4 Model Scale (Llama-3.2-3B)

We evaluate on Llama-3.2-3B to test whether the method adapts to model scale (Table 5).

The b^* distributions are statistically indistinguishable (KS test $p = 0.87$) because b^* is derived solely from q-score statistics, which depend on the data and detector, not the training model. The PPL improvement for 3B is minimal: 0.01 absolute (16.19 vs. 16.20), 0.06% relative—within noise.

This reveals a fundamental property: adaptive b^* provides a *data-driven* default that is consistent across models. This consistency is both a strength (predictable behavior) and a limitation (no model-specific adaptation). Drayson et al.'s observation that different model scales need different b values (1 vs. 10) suggests that optimal b is model-dependent, a dimension our q-score-only derivation cannot capture.

5.5 ESS Constraint Sensitivity ($n_{\min}$)

Table 6 presents the sensitivity of adaptive b^* to the ESS lower bound $n_{\min}$ (batch size $n = 4$).

The sensitivity curve reveals a narrow effective range. At $n_{\min} = 1$, the ESS constraint is too permissive: b^* reaches median 2.60, contaminated samples receive weight $w \approx 0.004$, and PPL (17.71) is *worse* than uniform (17.69) due to over-downweighting. At $n_{\min} \geq 3$, the constraint is too tight: b^* is clamped near zero, and the method degrades to uniform training. At $n_{\min} = 4 = n$ (batch size), $b^* = 0$ identically—the ESS constraint demands all weights be equal, which is only possible when $b = 0$.

Only $n_{\min} = 2$ occupies the effective region. While this single-point sensitivity is a limitation, we note that $n_{\min}$ selection reduces from a continuous grid search over b (requiring ≥ 6 runs) to checking 2–3 integer values, representing a significant practical simplification.

5.6 Cross-Generator Robustness

In realistic web corpora, synthetic content originates from multiple generator families. We test this by generating contamination with Llama-3.2-1B while training Qwen3-0.6B (Table 7).

The TF-IDF + LR detector, trained only on Qwen3-0.6B-generated text, generalizes poorly to Llama-3.2-1B outputs (AUC drops from 0.939 to 0.891). The q-score distribution becomes diffuse

Table 5: b^* distribution comparison across model scales ($\rho = 0.4$, seed 42). The b^* distribution is nearly identical because it depends only on q-score statistics, not model architecture. KS test $p = 0.87$.

Model	b^* med.	b^* mean	b^* P25	b^* P75	Δ_{rel} (%)
Qwen3-0.6B	0.87	1.40	0.54	1.51	0.23
Llama-3.2-3B	0.92	1.38	0.55	1.48	0.06

Table 6: $n_{\min}$ sensitivity ($\rho = 0.4$, Qwen3-0.6B, seed 42, batch size 4). Only $n_{\min} = 2$ improves over uniform; $n_{\min} = 1$ is too aggressive, $n_{\min} \geq 3$ collapses to uniform.

$n_{\min}$	Eval PPL	b^* median	Status
1	17.710	2.60	Worse than uniform
2	17.650	0.87	Best
3	17.690	$\sim$0.3	$\approx$ Uniform
4	17.690	0.0	= Uniform
5	17.690	0.0	= Uniform

(mid-range ratio rises from 0.463 to 0.52), degrading both the $\hat{\alpha}_{\text{eff}}$ estimates and the ranking quality of weights. Adaptive b^* (PPL = 17.77) is *worse* than uniform (17.75): noisy reweighting does more harm than good.

This exposes a fundamental dependency: adaptive b^* quality is bounded by detector signal-to-noise ratio. In web corpora with diverse synthetic sources, detector AUC may be lower than in controlled settings, making the method less applicable precisely where it would be most needed.

5.7 Domain-Skewed Injection

We test robustness to non-uniform contamination by concentrating 85% of synthetic content in the code domain.

Domain skew creates a bimodal batch structure: code-heavy batches have uniformly high q-scores ($\hat{\alpha}_{\text{eff}}$ overestimates, b^* too large), while text batches have low q-scores ($b^* \approx 0$). The per-batch mechanism, designed to adapt to local contamination variation, instead amplifies the skew by over-downweighting code batches that may contain useful programming knowledge.

5.8 Baseline Comparisons

Table 9 consolidates all methods under the standard setting ($\rho = 0.4$, Qwen3-0.6B, seed 42).

The ranking is: gentle reweighting ($b = 0.2$) $\gg$ adaptive $\gg$ detect-and-remove $>$ uniform $\approx$ random. Random reweighting (PPL = 17.695 $\approx$ uniform) confirms that detector-informed signal is load-bearing: arbitrary weight perturbations do not help. Detect-and-remove (PPL = 17.68) removes $\sim$42% of data (40% true synthetic + 2% false positives), losing useful signal that gentle reweighting preserves.

5.9 Downstream Evaluation

Table 10 reports MMLU and HellaSwag results for Llama-3.2-3B checkpoints.

No significant downstream differences are observed. MMLU performance (25.8%) is near random-chance level (25%), reflecting severe undertraining at 200M tokens for a 3B model (63 tokens/parameter

Table 7: Cross-generator evaluation ($\rho = 0.4$, Qwen3-0.6B, seed 42). When detector quality degrades on out-of-distribution synthetic text, adaptive reweighting becomes harmful.

Setting	Uniform	Adaptive	Δ	Verdict
Self-gen (Qwen $\rightarrow$ Qwen)	17.690	17.650	-0.040	Adaptive wins
Cross-gen (Llama $\rightarrow$ Qwen)	17.750	17.770	$+0.020$	Adaptive **loses**

Table 8: Domain-skewed contamination ($\rho = 0.4$, Qwen3-0.6B). Per-batch adaptation amplifies domain skew rather than correcting it.

Method	Uniform PPL	Adaptive PPL	Δ
Uniform injection	17.690	17.650	-0.040
Domain-skewed (code-heavy)	17.730	17.760	$+0.030$

versus $\sim$20$\times$ Chinchilla-optimal). The PPL improvement of 0.06% relative is far below the resolution of discrete accuracy benchmarks ($\sim$0.5% absolute). We note that PPL is a continuous metric with higher sensitivity than discrete accuracy, and do not claim downstream transfer at current scale.

6 Discussion

Why adaptive b^* overshoots. The $\hat{\alpha}_{\text{eff}}(b)$ framework estimates contamination from q-score statistics and seeks the most aggressive downweighting permitted by the ESS constraint. This objective is misaligned with the true optimum in our setting: FineWeb synthetic data generated by Qwen3-0.6B is not pure noise but retains substantial linguistic quality. The optimal strategy ($b = 0.2$, $w = 0.63$) is *mild correction*, not aggressive removal. The ESS constraint prevents catastrophic over-downweighting (cf. $n_{\min} = 1$, Table 6) but does not guide b^* toward the shallow optimum.

Diagnostic value of b^* trajectories. Despite numerical inaccuracy, b^* trajectories provide information absent from the Drayson framework. The shift from $b^* = 0.87$ ($\rho = 0.4$) to $b^* = 1.08$ ($\rho = 0.6$) correctly indicates increased contamination. Per-batch b^* variance reveals contamination heterogeneity: batches with higher contamination receive larger b^*, making the framework interpretable for data auditing even when not producing optimal weights.

When adaptive hurts. We identify two failure modes: (1) cross-generator contamination, where detector quality degrades (AUC 0.939 $\rightarrow$ 0.891) and noisy q-scores lead to harmful reweighting (Section 5.6); and (2) domain-skewed contamination, where per-batch adaptation amplifies rather than corrects the skew (Section 5.7). Both failure modes stem from violations of implicit assumptions: the detector generalizes well (violated in cross-generator), and contamination is roughly uniform across batches (violated in domain skew).

The $n_{\min}$ sensitivity question. A reviewer might argue that replacing grid search over b with selection of $n_{\min}$ merely substitutes one hyperparameter for another. We partially concede: the method is not hyperparameter-free. However, $n_{\min}$ is an integer in $\{1, \ldots, n\}$ (typically 2–3 effective values), requiring far fewer runs to sweep than $b \in [0.1, 10]$ (our grid uses 9 values). This represents a reduction in tuning cost, not its elimination.

Table 9: Method comparison ($\rho = 0.4$, Qwen3-0.6B, seed 42). Grid-best $b = 0.2$ dominates; adaptive provides partial improvement; random reweighting confirms detector signal is necessary.

Method	PPL	Δ vs. Uniform	Signal source
Grid-best ($b = 0.2$)	17.637	-0.053	Oracle grid
Adaptive b^*	17.650	-0.040	Adaptive q-based
Detect-and-remove ($q > 0.5$)	17.680	-0.010	Hard threshold
Uniform	17.690	0.000	None
Random reweighting	17.695	$+0.005$	Random

Table 10: Downstream evaluation (Llama-3.2-3B, $\rho = 0.4$, seed 42, 200M tokens). No significant differences are observed, consistent with the minimal PPL gap (0.06% relative) and severe undertraining (63 tokens/parameter vs. Chinchilla-optimal $\sim$20$\times$).

Benchmark	Uniform	Adaptive	Δ
MMLU (5-shot)	25.8%	25.8%	0.0%
HellaSwag (10-shot)	38.2%	38.3%	+0.1%

Loss reweighting vs. SIR. Loss reweighting enables per-batch adaptive b^*, which is architecturally incompatible with SIR's per-epoch resampling. The continuous gradient-level control also avoids the discrete variance introduced by stochastic resampling. A full gradient variance analysis under both implementations remains future work.

7 Limitations

Scale. All experiments use 200M tokens, far below production pretraining scale. The 3B model is severely undertrained (63 tokens/parameter). Effect sizes at scale may differ qualitatively.

Generator diversity. Cross-generator results (Section 5.6) demonstrate degradation with a single out-of-distribution generator. Real web corpora contain outputs from many generators and model families; robustness under such diversity is untested.

Contamination type. We study continuation-based synthetic text. Other contamination types (paraphrase, translation, summarization, instruction-following outputs) may interact differently with provenance detectors and reweighting schemes.

$n_{\min}$ **sensitivity.** The narrow effective range of $n_{\min}$ at batch size 4 may not generalize to larger batch sizes, where the integer space grows and the ESS constraint landscape changes.

8 Conclusion

We presented adaptive b^*, an online method for deriving the weighting exponent in provenance-weighted pretraining from per-batch contamination statistics. Our investigation yields three actionable findings: (1) lightweight detectors (TF-IDF + LR) suffice for provenance weighting, with 3000$\times$ fewer parameters and negligible PPL difference compared to neural alternatives; (2) the $\hat{\alpha}_{\text{eff}}(b)$ framework provides batch-level contamination diagnostics, though its derived b^* systematically overshoots the grid optimum by $\sim$4$\times$; and (3) adaptive b^* serves as a zero-cost reasonable default recovering $\sim$70% of grid-best improvement, while failing under cross-generator and domain-skewed contamination. These results suggest that future work should focus on closing the overshoot

gap—potentially through model-aware corrections to b^*—and on improving detector robustness to diverse synthetic sources at scale.

References

Drayson, O., et al. Detecting and mitigating model-generated text in internet corpora. In *EMNLP*, 2025.

Shumailov, I., et al. AI models collapse when trained on recursively generated data. In *Nature*, 2024.

Dohmatob, E., et al. Strong model collapse. In *ICLR*, 2025.

Xie, S. M., et al. Data selection for language models via importance resampling. In *NeurIPS*, 2023.

Xie, S. M., et al. DoReMi: Optimizing data mixtures by domain reweighting. In *NeurIPS*, 2023.

Abbas, A., et al. SoftDedup: Soft deduplication for training data. In *ACL*, 2024.

Li, Y., et al. Golden ratio weighting for synthetic and human data. *arXiv:2502.18049*, 2025.

Anonymous. Convergence bounds for contaminated distributions. *arXiv:2602.16065*, 2025.

Okanovic, D., et al. Dynamic loss-based reweighting for pretraining. In *ICLR*, 2025.

Hoffmann, J., et al. Training compute-optimal large language models. In *NeurIPS*, 2022.